

Full-Stack App — Deployment Plan v5 (Generic Template)

Django backend (server) + Electron/Next.js frontend (client PCs), LAN-deployed, Nuitka-compiled, licensed.

Backend and frontend are **two separate projects** with **two separate installers**. The backend ships once to a single server machine; the frontend ships as a one-click Electron app to every workstation. Each frontend client reads its own runtime config (including the backend's LAN address) from a user-editable file that lives **outside** the built app.

This version is a reusable template, not tied to any one product. Everywhere you see `app-server`, `app-client`, `AppClient`, `AppServer`, or `com.yourcompany.appclient`, swap in your actual project's name. There is nothing clinic-specific left in this plan — it applies to any Django + Electron/Next.js LAN deployment with offline licensing.

What changed from v4

Reordered (two real bugs fixed, not just cosmetic):

- **Logging now comes before Licensing.** v4 built the license system's "clear error, not silent crash" requirement (Task on license failures) before logging infrastructure existed to actually write `license.log`. You can't log a license failure to a file that isn't configured yet.
- **Compiling with Nuitka now comes before setting up the OS service.** v4's systemd unit and NSSM commands pointed at `backend.bin` / `backend.exe` — files that only exist *after* Nuitka compiles them in a later phase. Wiring a service to a binary that doesn't exist yet doesn't work. Compile first, then wrap it in a service.
- Backup Strategy now sits after the service is running, since in practice you want to back up a deployment that's already live under systemd/NSSM, not a dev-mode process.

Filled in (previously just described, now actually written):

- Full Django LOGGING config (Phase 3), including the `license` logger referenced but never defined in v4.
- `ALLOWED_HOSTS` and `CORS_ALLOWED_ORIGINS` now actually read from `.env` in `settings.py` — v4's Task 1.5 said these live in `.env`, but v4's Task 2.3 code hardcoded them directly in `settings.py`, contradicting itself.
- `collectstatic` settings code and an Nginx config for serving `/media/` (Phase 2) — v4 only said "do this," never showed how.
- Firewall commands for Linux (`ufw/firewalld`) and Windows (`netsh`) (Phase 2 and Phase 6).
- Full systemd unit file and full NSSM command sequence, not fragments (Phase 6).

- `backup.sh` and `backup.bat`, referenced by filename in v4 but never written (Phase 7).
- The HWID hand-off workflow between you and the client machine — v4 had a script to generate a HWID and a script to consume one, but no described process connecting them (Phase 4).
- `license-invalid.html` — v4's Electron main process calls `errWindow.loadFile('license-invalid.html')`, but that file was never written.
- A working retry path — v4's "Retry" button had no handler; retrying just reloaded a static error page forever. Added a `retry-license-check` IPC round trip that actually re-checks and opens the main window on success.
- `contextIsolation/nodeIntegration` flags were set on the main window in v4 but silently missing from the error window — fixed for consistency.

Added:

- A new closing section, **Future Feature — License Renewal Endpoint**, covering how to let a license be renewed/updated without someone manually swapping `license.lic` on disk and restarting the service.

Index

1. [Phase 1 — Two Project Structures \(Server vs. Client\)](#)
 2. [Phase 2 — Make the Backend Production Ready](#)
 3. [Phase 3 — Logging](#)
 4. [Phase 4 — Backend Licensing](#)
 5. [Phase 5 — Compile Backend With Nuitka](#)
 6. [Phase 6 — Windows Service / Linux Service](#)
 7. [Phase 7 — Backup Strategy](#)
 8. [Phase 8 — Electron Frontend With Runtime Config](#)
 9. [Phase 9 — Frontend Licensing Gate in Electron](#)
 10. [Phase 10 — Two Installers \(Server + Client\)](#)
 11. [Phase 11 — Security Hardening \(Audit Pass\)](#)
 12. [Milestone Order](#)
 13. [Future Feature — License Renewal Endpoint](#)
-

Phase 1 — Two Project Structures (Server vs. Client)

Goal: separate the deployment artifacts for the server (backend) and client PCs (frontend) from day one. They will never ship together.

Task 1.1 — Backend (Server) Deployment Layout

```
app-server/
├── backend/          # compiled backend binary (Nuitka output)
├── config/           # .env, license.lic, public_key.pem
├── data/             # sqlite database
├── media/            # uploaded files
├── logs/             # backend logs
├── backups/          # zip archives produced by backup.sh / backup.bat
└── scripts/          # startup / backup / hwid scripts

backend/  -> executable files (backend.exe / backend.bin)
config/   -> .env, license.lic, public_key.pem
data/     -> db.sqlite3
media/    -> uploaded files
logs/     -> logs
backups/  -> backup_YYYY-MM-DD_HHMMSS.zip
scripts/  -> backup.sh / backup.bat / get_machine_id.py
```

Task 1.2 — Frontend (Client) Deployment Layout

```
app-client/
├── app/              # Electron + Next.js packaged app
│   ├── AppClient.exe # the Electron launcher
│   └── resources/
│       ├── app.asar   # packaged Next.js + Electron code
│       ├── default.env # template copied to userData on first run
│       └── ... (chrome runtime, etc.)
└── (per-user runtime data — NOT shipped)
    ├── %APPDATA%/AppClient/
    │   ├── config.env # <-- USER-EDITABLE, lives outside the .exe
    │   └── logs/
```

The user-editable file is **config.env**, stored in `app.getPath('userData')` on each client PC. This is what makes variables updatable after build — see Phase 8 for the mechanism.

Task 1.3 — Move SQLite Outside Backend Code

```
backend/db.sqlite3 -> app-server/data/db.sqlite3
```

Task 1.4 — Move Media Outside Backend Code

```
backend/media/ -> app-server/media/
```

Task 1.5 — Move Backend .env Outside Code

```
backend/.env -> app-server/config/.env
```

Backend **.env** carries environment-specific values only — **not** licensing enforcement settings (those live inside the signed license token; see Phase 4):

```
SECRET_KEY=change-me
DEBUG=False
DATABASE_URL=sqlite:///data/db.sqlite3

# server
PORT=8000
ALLOWED_HOSTS=192.168.1.100,localhost,127.0.0.1

# CORS — must include the Electron app's origin (file:// or
http://localhost:<port>)
CORS_ALLOWED_ORIGINS=http://localhost:3000,app://app-client
```

config/license.lic and config/public_key.pem are fixed, hardcoded filenames under CONFIG_DIR — no env var needed for them.

Path resolution: resolve every path from one explicit root, not `__file__`. Once compiled with Nuitka --onefile, `__file__` and `sys.argv[0]` resolve to a temporary extraction folder, not the real app-server/ directory.

```
# settings.py / run_server.py
import os
DEPLOY_ROOT = os.environ.get("DEPLOY_ROOT") or os.getcwd()
CONFIG_DIR = os.path.join(DEPLOY_ROOT, "config")
DATA_DIR = os.path.join(DEPLOY_ROOT, "data")
MEDIA_DIR = os.path.join(DEPLOY_ROOT, "media")
LOG_DIR = os.path.join(DEPLOY_ROOT, "logs")
BACKUP_DIR = os.path.join(DEPLOY_ROOT, "backups")
```

Set DEPLOY_ROOT explicitly wherever the backend process starts (dev shell, systemd Environment=, NSSM "Startup directory").

Load .env explicitly from CONFIG_DIR:

```
from dotenv import load_dotenv
load_dotenv(os.path.join(CONFIG_DIR, ".env"))
```

Permissions (do this now, not at the end):

```
mkdir -p app-server/logs app-server/backups
chmod 600 app-server/config/.env
chown app-svc:app-svc -R app-server/config
```

STOP

```
✓ two separate project trees exist (app-server/, app-client/)
✓ backend sqlite/media/.env outside code, chmod 600, owned by service account
✓ DEPLOY_ROOT resolves correctly when launched from a different working
directory
✓ client userData path identified as the home of the runtime config.env
```

Phase 2 — Make the Backend Production Ready

Task 2.1 — Install Dependencies

```
pip install waitress python-dotenv whitenoise django-cors-headers
```

`django-cors-headers` is now **required** — the Electron app on a different machine is a cross-origin client.

Task 2.2 — Static Files (Whitenoise)

```
# settings.py
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware",
    # ...rest of your middleware
]

STATIC_URL = "/static/"
STATIC_ROOT = os.path.join(DEPLOY_ROOT, "staticfiles")
STATICFILES_STORAGE = "whitenoise.storage.CompressedManifestStaticFilesStorage"
```

Run once per deploy (and again after every code update):

```
python manage.py collectstatic --noinput
```

Task 2.3 — Media Files

Two options for serving `/media/`:

Option A — Nginx reverse proxy (recommended for LAN deployments):

```
# /etc/nginx/sites-available/app-server
server {
    listen 80;
    server_name app-server.local;

    location /static/ {
        alias /opt/app/app-server/staticfiles/;
    }

    location /media/ {
        alias /opt/app/app-server/media/;
    }

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

sudo ln -s /etc/nginx/sites-available/app-server /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

Option B — Authenticated Django view streaming files from `MEDIA_DIR`, if you need per-file access control instead of a flat proxy. Skip Nginx entirely for a first LAN rollout if you don't need

HTTPS yet — add it in Phase 11.

Task 2.4 — CORS and Allowed Hosts, Read From .env

This is the part v4 got wrong — the code has to actually read the values Task 1.5 put in .env, not hardcode them:

```
# settings.py
import os

ALLOWED_HOSTS = [h.strip() for h in os.environ.get("ALLOWED_HOSTS",
"").split(",") if h.strip()]

CORS_ALLOWED_ORIGINS = [
    o.strip() for o in os.environ.get("CORS_ALLOWED_ORIGINS", "").split(",") if
o.strip()
]
CORS_ALLOW_CREDENTIALS = True
```

Task 2.5 — Create Startup File

```
# run_server.py
import os
from dotenv import load_dotenv

DEPLOY_ROOT = os.environ.get("DEPLOY_ROOT") or os.getcwd()
load_dotenv(os.path.join(DEPLOY_ROOT, "config", ".env"))

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")

from waitress import serve
from config.wsgi import application

port = int(os.environ.get("PORT", 8000))
serve(application, host="0.0.0.0", port=port)
```

Task 2.6 — Open the Firewall

```
# Linux — ufw
sudo ufw allow 8000/tcp

# Linux — firewallld
sudo firewall-cmd --add-port=8000/tcp --permanent
sudo firewall-cmd --reload

# Windows
netsh advfirewall firewall add rule name="App Server API" dir=in action=allow
protocol=TCP localport=8000
```

Task 2.7 — Test From Another PC

```
python run_server.py
```

From a second machine, hit `http://SERVER_IP:PORT`. Verify the API works from another machine, not just localhost.

STOP

- ✓ another PC can reach the API on the port defined in .env
- ✓ CORS headers present on responses (Electron client won't be blocked)
- ✓ ALLOWED_HOSTS and CORS_ALLOWED_ORIGINS are read from .env, not hardcoded
- ✓ collectstatic has run and static assets load correctly
- ✓ firewall rule for PORT confirmed on the actual OS you're deploying to

Phase 3 — Logging

Moved ahead of Licensing on purpose: the licensing phase needs a working `license.log` from the start, not a promise of one added later.

app-server/logs/

```
# settings.py
LOGGING = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {
        "verbose": {
            "format": "{asctime} {levelname} {module} {message}",
            "style": "{",
        },
    },
    "handlers": {
        "app_file": {
            "level": "INFO",
            "class": "logging.handlers.RotatingFileHandler",
            "filename": os.path.join(LOG_DIR, "application.log"),
            "maxBytes": 5 * 1024 * 1024,
            "backupCount": 5,
            "formatter": "verbose",
        },
        "error_file": {
            "level": "ERROR",
            "class": "logging.handlers.RotatingFileHandler",
            "filename": os.path.join(LOG_DIR, "errors.log"),
            "maxBytes": 5 * 1024 * 1024,
            "backupCount": 5,
            "formatter": "verbose",
        },
        "license_file": {
            "level": "INFO",
            "class": "logging.handlers.RotatingFileHandler",
            "filename": os.path.join(LOG_DIR, "license.log"),
            "maxBytes": 2 * 1024 * 1024,
            "backupCount": 3,
            "formatter": "verbose",
        },
    },
    "loggers": {
        "django": {
            "handlers": ["app_file", "error_file"],
            "level": "INFO",
            "propagate": True,
        },
        "license": {
            "handlers": ["license_file"],
```

```
        "level": "INFO",
        "propagate": False,
    },
}
```

This gives Phase 4's license module a `license` logger to write to, and gives support staff a dedicated file — license failures are the #1 support call, and they shouldn't be buried in `application.log`.

STOP

```
✓ application.log, errors.log, license.log all write to app-server/logs/
✓ log rotation confirmed working (maxBytes/backupCount honored)
```

Phase 4 — Backend Licensing

Goal: the backend only runs for machines you've issued a license to, works offline by default, and can optionally phone home for centralized control.

Licensing model: the license is bound to the **server machine** only (one license per deployment). Client PCs do **not** carry their own license — they simply refuse to connect to a backend that reports an invalid license. This keeps license issuance simple while still letting you revoke the whole deployment at once.

Task 4.1 — Design the License Token

Use a signed JWT (RS256):

```
pip install pyjwt cryptography

{
  "client": "Customer / Deployment Name",
  "license_id": "uuid",
  "hwid": "sha256 of machine fingerprint",
  "features": ["billing", "reports"],
  "max_clients": 10,
  "mode": "offline",
  "server_url": "https://license.yourcompany.com/api/validate",
  "grace_days": 14,
  "iat": 1751328000,
  "exp": 1782864000
}
```

`max_clients` is informational — actual concurrent-client enforcement is a backend concern (count distinct IPs/sessions, return 403 if exceeded).

Task 4.2 — Generate a Signing Keypair (you keep this, once)

```
openssl genrsa -out private_key.pem 2048
openssl rsa -in private_key.pem -pubout -out public_key.pem
```

- `private_key.pem` — stays on your machine, never shipped.
- `public_key.pem` — ships with the backend, goes in `app-server/config/public_key.pem`.

Task 4.3 — Machine Fingerprint Script

```
# scripts/get_machine_id.py
import hashlib, platform, uuid

def machine_id():
    raw = f"{uuid.getnode()}-{platform.node()}-{platform.system()}"
    return hashlib.sha256(raw.encode()).hexdigest()

if __name__ == "__main__":
    print(machine_id())
```

Task 4.3b — The HWID Hand-off Workflow (missing in v4)

This is the step v4 never spelled out: how the hash generated on the customer's machine actually gets into `issue_license.py` on yours.

1. During server install (or before shipping the server installer), run:

```
python scripts/get_machine_id.py
```

2. Whoever is on-site (you, IT, or the installer itself — see Phase 10) copies the printed hash and sends it to you (email, ticket, or a "Copy HWID" button if you build one into the server installer's first-run screen).
3. You paste that hash into `issue_license.py`'s `hwid` field and run it (Task 4.4).
4. You send back the resulting `license.lic` file for them to place in `config/license.lic`.
5. If the server's hardware changes later (new NIC, reimage), the HWID changes too — repeat this loop to issue a replacement license. (See the Future Feature section for a way to do this without a manual file swap.)

Task 4.4 — Issue a License (your side, offline tool, not shipped)

```
# scripts/issue_license.py (keep on your machine, never distribute)
import jwt, time

payload = {
    "client": "Customer / Deployment Name",
    "license_id": "deploy-001",
    "hwid": "<paste from get_machine_id.py>",
    "features": ["billing", "reports"],
    "max_clients": 10,
    "mode": "offline",
    "server_url": "https://license.yourcompany.com/api/validate",
    "grace_days": 14,
    "iat": int(time.time()),
    "exp": int(time.time()) + 60 * 60 * 24 * 365,
```

```

}

with open("private_key.pem") as f:
    private_key = f.read()

token = jwt.encode(payload, private_key, algorithm="RS256")
with open("license.lic", "w") as f:
    f.write(token)

```

Send the client `license.lic` → they drop it in `app-server/config/license.lic`,
`chmod 600`, owned by the service account.

Task 4.5 — Backend Validation Module

```

# license_check.py
import jwt, hashlib, platform, uuid, os, logging

logger = logging.getLogger("license")

def current_hwid():
    raw = f"{uuid.getnode()}-{platform.node()}-{platform.system()}"
    return hashlib.sha256(raw.encode()).hexdigest()

def get_public_key(config_dir):
    with open(os.path.join(config_dir, "public_key.pem")) as f:
        return f.read()

def validate_license_token(token, public_key):
    return jwt.decode(token, public_key, algorithms=["RS256"])

def validate_license(config_dir):
    public_key = get_public_key(config_dir)
    try:
        with open(os.path.join(config_dir, "license.lic")) as f:
            token = f.read().strip()
    except FileNotFoundError:
        logger.warning("License check failed: missing license.lic")
        raise ValueError("missing")

    try:
        payload = validate_license_token(token, public_key)
    except jwt.ExpiredSignatureError:
        logger.warning("License check failed: expired")
        raise ValueError("expired")
    except jwt.InvalidSignatureError:
        logger.warning("License check failed: tampered")
        raise ValueError("tampered")

    if payload["hwid"] != current_hwid():
        logger.warning("License check failed: hwid_mismatch")
        raise ValueError("hwid_mismatch")

    payload.setdefault("mode", "offline")
    payload.setdefault("grace_days", 14)
    logger.info(f"License check passed for {payload['client']}")
    (exp={payload['exp']})
    return payload

```

Wire this into Django as a startup check + a lightweight cached middleware, and expose a
`/api/license/status/` endpoint that returns:

```
{ "valid": true, "client": "...", "exp": 1782864000, "features":  
["billing","reports"] }
```

or

```
{ "valid": false, "reason": "expired" }
```

with reason being one of missing, expired, hwid_mismatch, tampered. This endpoint is what the Electron client will call on boot (Phase 9) — and later, what the renewal endpoint in the Future Feature section will refresh in place.

Task 4.6 — Optional Online Mode

If `payload["mode"] == "online"`, add a periodic check that calls `payload["server_url"]` with `license_id + hwid`. Use `grace_days` so flaky connections don't lock the deployment out immediately. (This is the same mechanism the Future Feature section builds on for auto-renewal.)

Task 4.7 — Test Backend Licensing

```
✓ valid license → app runs normally  
✓ missing license file → clear error, logged to license.log  
✓ expired license → clear error, logged  
✓ license for wrong hwid → clear error, logged  
✓ tampered license file → rejected (signature check fails), logged  
✓ (online mode) revoked license → blocked after grace period  
✓ (online mode) offline for < grace period → still runs  
✓ /api/license/status/ returns correct JSON for each state
```

STOP

```
✓ backend refuses to run without a valid, matching license  
✓ /api/license/status/ exposes a clean JSON status the Electron client can  
consume  
✓ every failure path writes a line to logs/license.log
```

Phase 5 — Compile Backend With Nuitka

Only now — after paths, static/media serving, logging, and licensing are all proven to work uncompiled. Moved ahead of the OS service step (Phase 6), because that step's config files point directly at the binary this phase produces.

```
pip install nuitka  
  
python -m nuitka \  
    --standalone \  
    --onefile \  
    --include-package=<your_django_app_1> \  
    --include-package=<your_django_app_2> \  
    --include-package-data=django \  
    --enable-plugin=anti-bloat \  
    --enable-plugin=py-sqlite3
```

```
--enable-plugin=data-files \
run_server.py
```

Notes:

- `--include-package=` for **each Django app** is mandatory.
- `data-files` pulls in Django admin templates/static.
- If uploaded images go through Pillow: `--include-package=PIL`.
- **Compile on the same OS you're deploying to** — Nuitka does not cross-compile. Build the Windows binary on Windows, the Linux binary on Linux.
- **Windows toolchain:** Nuitka needs either MSVC or MinGW64 present. If neither is installed, add `--assume-yes-for-downloads` to let Nuitka fetch a MinGW64 toolchain automatically, or install Visual Studio Build Tools first and pass `--msvc=latest`.
- Build inside a clean virtualenv matching the exact Python version you'll run in production — mismatched versions are the most common source of "works on my machine, fails on theirs."

Test the Compiled Binary

The binary still reads, from outside:

```
config/.env
config/license.lic
config/public_key.pem
data/db.sqlite3
media/
```

Re-run the entire Task 4.7 license lifecycle test against the compiled binary — path-resolution bugs surface here, not in the uncompiled version.

STOP — Backend binary ready

```
✓ compiled backend binary runs standalone from a plain shell (no service wrapper yet)
✓ reads config/data/media from outside the binary
✓ full license lifecycle (Task 4.7) re-tested against the compiled binary
✓ logging still writes correctly from the compiled binary
```

Phase 6 — Windows Service / Linux Service

This now correctly points at the compiled binary from Phase 5, not source code.

Linux (systemd)

```
# /etc/systemd/system/app.service
```

```
[Unit]
Description=App Server Backend
After=network.target

[Service]
Type=simple
User=app-svc
Group=app-svc
WorkingDirectory=/opt/app/app-server
Environment=DEPLOY_ROOT=/opt/app/app-server
ExecStart=/opt/app/app-server/backend/backend.bin
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable app.service
sudo systemctl start app.service
sudo systemctl status app.service
```

Windows (NSSM)

```
nssm install AppServer "C:\AppServer\backend\backend.exe"
nssm set AppServer AppDirectory "C:\AppServer"
nssm set AppServer AppEnvironmentExtra DEPLOY_ROOT=C:\AppServer
nssm set AppServer Start SERVICE_AUTO_START
nssm set AppServer AppStdout C:\AppServer\logs\service-stdout.log
nssm set AppServer AppStderr C:\AppServer\logs\service-stderr.log
nssm set AppServer ObjectName .\app-svc <service-account-password>
nssm start AppServer
```

Run the service under a low-privilege dedicated account (app-svc), not LocalSystem or an admin account.

Verify

```
# Linux
sudo reboot
# after reboot:
systemctl status app.service
curl http://localhost:8000/api/license/status/

# Windows
shutdown /r /t 0
# after reboot:
nssm status AppServer
Invoke-RestMethod http://localhost:8000/api/license/status/
```

STOP

- ✓ server reboot → backend service starts automatically → license still validates correctly
- ✓ service runs under a dedicated low-privilege account, not admin/root/LocalSystem
- ✓ stdout/stderr from the service land somewhere you can read (service-stdout.log)

```
/ journalctl)
```

Phase 7 — Backup Strategy

Backup ONLY:

```
app-server/config/    (includes .env AND license.lic)
app-server/data/
app-server/media/
```

Never backup:

```
app-server/backend/    # regenerate from source + Nuitka, don't archive binaries
app-client/            # client PCs aren't backed up from the server side
```

scripts/backup.sh (Linux)

```
#!/usr/bin/env bash
set -euo pipefail

DEPLOY_ROOT="/opt/app/app-server"
BACKUP_DIR="$DEPLOY_ROOT/backups"
DATE=$(date +%Y-%m-%d_%H%M%S)
ARCHIVE="$BACKUP_DIR/backup_${DATE}.zip"

mkdir -p "$BACKUP_DIR"

zip -r -e "$ARCHIVE" \
    "$DEPLOY_ROOT/config" \
    "$DEPLOY_ROOT/data" \
    "$DEPLOY_ROOT/media" \
    -x "*.pyc"

chmod 600 "$ARCHIVE"
echo "Backup created at $ARCHIVE"

# Prune backups older than 30 days
find "$BACKUP_DIR" -name "backup_*.zip" -mtime +30 -delete

chmod +x scripts/backup.sh
```

`zip -e` prompts for a password interactively; for unattended cron runs, either pipe a password via `-P` (weaker — visible in process list) or switch to `age/gpg` for the archive step if that's a concern.

scripts/backup.bat (Windows)

```
@echo off
setlocal enabledelayedexpansion

set DEPLOY_ROOT=C:\AppServer
set BACKUP_DIR=%DEPLOY_ROOT%\backups

if not exist "%BACKUP_DIR%" mkdir "%BACKUP_DIR%"

for /f %i in ('powershell -NoProfile -Command "Get-Date -Format yyyy-MM-dd_HH:mm:ss"') do set STAMP=%i
```

```
powershell -NoProfile -Command ^
  "Compress-Archive -Path '%DEPLOY_ROOT%\config', '%DEPLOY_ROOT%
\data', '%DEPLOY_ROOT%\media' -DestinationPath '%BACKUP_DIR%\backup_%STAMP%.zip'
-Force"

echo Backup created: %BACKUP_DIR%\backup_%STAMP%.zip

REM Prune backups older than 30 days
forfiles /p "%BACKUP_DIR%" /m backup_*.zip /d -30 /c "cmd /c del @path" 2>nul
```

Compress-Archive doesn't support password protection. If encryption is required, use 7-Zip instead:

```
"C:\Program Files\7-Zip\7z.exe" a -tzip -p%BACKUP_PASSWORD% "%BACKUP_DIR%
\backup_%STAMP%.zip" "%DEPLOY_ROOT%\config" "%DEPLOY_ROOT%\data" "%DEPLOY_ROOT%
\media"
```

Scheduling

```
# crontab -e (Linux)
0 2 * * * /opt/app/app-server/scripts/backup.sh >>
/opt/app/app-server/logs/backup.log 2>&1

# Windows Task Scheduler
schtasks /create /tn "AppServerBackup" /tr "C:\AppServer\scripts\backup.bat" /sc
daily /st 02:00
```

The archive contains SECRET_KEY and license.lic — keep owner-only permissions and encrypt before copying offsite.

STOP

```
✓ backup.sh / backup.bat run manually once and produce a valid archive
✓ scheduled job confirmed to fire (check logs/backup.log or Task Scheduler
history)
✓ old backups get pruned instead of filling the disk
✓ a restored backup (config/ + data/ + media/ dropped back in) brings the
service back to a working state
```

Phase 8 — Electron Frontend With Runtime Config

The frontend is an **Electron desktop app** that:

1. Gets packaged into a one-click installer (AppClient-Setup.exe).
2. Runs on each workstation independently.
3. Connects to the backend over the LAN at whatever address is configured in a **user-editable config.env** file that lives outside the packaged .asar.

Task 8.1 — Why Standard `NEXT_PUBLIC_*` Won't Work

`NEXT_PUBLIC_*` variables are **inlined at build time** by Next.js. Once you run `next build`, a string like `http://192.168.1.100:8000` is permanently baked into the JS bundle — there's no `.env` to edit post-build, because the bundle has no idea one exists.

For a deployment where every workstation may point at a different (or changing) server IP, and you don't want to rebuild and redistribute for every IP change, use a **runtime config** approach instead.

Task 8.2 — Runtime Config: `config.env` in `userData`

The Electron main process loads `config.env` from `app.getPath('userData')` at startup, then exposes the values to the Next.js renderer via IPC.

Why `userData` (not next to the `.exe`):

- `userData` on Windows is `%APPDATA%\AppClient\` — **user-writable without admin rights**.
- Next to the `.exe` is usually `C:\Program Files\AppClient\` — requires admin to edit.
- `userData` survives app updates (electron-updater replaces Program Files, leaves AppData alone).

File layout on the client machine after install:

```
C:\Users\<user>\AppData\Roaming\AppClient\  
├─ config.env  
└─ logs\
```

Template `default.env` (shipped inside `resources/`):

```
# Backend connection – edit this if the server IP changes, then restart the app  
API_URL=http://192.168.1.100:8000  
  
# Optional feature flags the renderer can read  
ENABLE_TELEMETRY=false  
DEFAULT_LANGUAGE=en
```

Task 8.3 — Electron Main Process: Load + Expose Config

```
// electron/main.js  
const { app, BrowserWindow, ipcMain } = require('electron');  
const path = require('path');  
const fs = require('fs');  
const dotenv = require('dotenv');  
  
let mainWindow;  
let runtimeConfig = {};  
  
function loadOrCreateConfig() {  
  const userDataPath = app.getPath('userData');  
  const configPath = path.join(userDataPath, 'config.env');  
  if (fs.existsSync(configPath)) {  
    dotenv.config({ path: configPath });  
    runtimeConfig = process.env;  
  } else {  
    fs.writeFileSync(configPath, '# Backend connection – edit this if the server IP changes, then restart the app\nAPI_URL=http://192.168.1.100:8000\n\n# Optional feature flags the renderer can read\nENABLE_TELEMETRY=false\nDEFAULT_LANGUAGE=en');  
    loadOrCreateConfig();  
  }  
}
```

```

    if (!fs.existsSync(configPath)) {
      const templatePath = path.join(process.resourcesPath, 'default.env');
      if (fs.existsSync(templatePath)) {
        fs.copyFileSync(templatePath, configPath);
      } else {
        fs.writeFileSync(configPath, 'API_URL=http://localhost:8000\n');
      }
    }

    const parsed = dotenv.config({ path: configPath }).parsed || {};
    runtimeConfig = parsed;
    return runtimeConfig;
  }

ipcMain.handle('get-config', () => runtimeConfig);

ipcMain.handle('set-config', (event, newValues) => {
  const userDataPath = app.getPath('userData');
  const configPath = path.join(userDataPath, 'config.env');
  const merged = { ...runtimeConfig, ...newValues };
  const serialized = Object.entries(merged).map(([k, v]) => `${k}=${v}`)
    .join('\n');
  fs.writeFileSync(configPath, serialized + '\n');
  runtimeConfig = merged;
  return merged;
});

async function checkBackendLicense(apiUrl) {
  try {
    const res = await fetch(`${apiUrl}/api/license/status/`, { signal:
AbortSignal.timeout(5000) });
    if (!res.ok) return { valid: false, reason: 'http_error' };
    return await res.json();
  } catch (e) {
    return { valid: false, reason: 'unreachable' };
  }
}

function openMainWindow() {
  mainWindow = new BrowserWindow({
    width: 1280,
    height: 800,
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
      contextIsolation: true,
      nodeIntegration: false,
    },
  });
  mainWindow.loadFile(path.join(__dirname, '../renderer/out/index.html'));
}

// IPC handler for the "Retry" button on the license-invalid screen
ipcMain.handle('retry-license-check', async () => {
  const cfg = loadOrCreateConfig();
  const apiUrl = cfg.API_URL || 'http://localhost:8000';
  const status = await checkBackendLicense(apiUrl);
  if (status.valid) {
    BrowserWindow.getAllWindows().forEach(w => w.close());
    openMainWindow();
  }
  return status;
});

async function startApp() {

```

```

const cfg = loadOrCreateConfig();
const apiUrl = cfg.API_URL || 'http://localhost:8000';
const status = await checkBackendLicense(apiUrl);

if (!status.valid) {
  const errWindow = new BrowserWindow({
    width: 480,
    height: 320,
    resizable: false,
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
      contextIsolation: true,
      nodeIntegration: false,
    },
  });
  errWindow.loadFile(path.join(__dirname, 'license-invalid.html'));
  errWindow.webContents.on('did-finish-load', () => {
    errWindow.webContents.send('license-error', status);
  });
  return;
}

openMainWindow();
}

app.whenReady().then(startApp);

```

Preload bridge:

```

// electron/preload.js
const { contextBridge, ipcRenderer } = require('electron');

contextBridge.exposeInMainWorld('appConfig', {
  get: () => ipcRenderer.invoke('get-config'),
  set: (newValues) => ipcRenderer.invoke('set-config', newValues),
  onLicenseError: (callback) => ipcRenderer.on('license-error', (_event, data)
=> callback(data)),
  retryLicenseCheck: () => ipcRenderer.invoke('retry-license-check'),
});

```

electron/license-invalid.html (missing from v4 — this is what errWindow.loadFile was pointing at with nothing there)

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8" />
  <title>License Problem</title>
  <style>
    body { font-family: system-ui, sans-serif; padding: 24px; background:
#1e1e1e; color: #f5f5f5; }
    h1 { font-size: 18px; margin-bottom: 8px; }
    #reason { color: #ff8080; margin: 12px 0 20px; line-height: 1.4; }
    button { padding: 8px 16px; cursor: pointer; }
  </style>
</head>
<body>
  <h1>Can't connect to a licensed server</h1>
  <p id="reason">Checking license status...</p>
  <button id="retry">Retry</button>

```

```

<script>
  const reasons = {
    missing: "No license file found on the server.",
    expired: "The server's license has expired.",
    hwid_mismatch: "The license doesn't match this server.",
    tampered: "The license file failed verification.",
    unreachable: "Can't reach the backend server. Check the server IP in
Settings.",
    http_error: "The backend server returned an error.",
  };

  function showReason(status) {
    document.getElementById('reason').textContent =
      reasons[status.reason] || `License problem: ${status.reason}`;
  }

  window.appConfig.onLicenseError(showReason);

  document.getElementById('retry').addEventListener('click', async () => {
    const status = await window.appConfig.retryLicenseCheck();
    if (!status.valid) {
      showReason(status);
    }
    // if valid, the main process closes this window and opens the app window
    itself
  });
</script>
</body>
</html>

```

Renderer-side usage (instead of `process.env.NEXT_PUBLIC_*`):

```

// lib/config.ts (runs in renderer)
let cachedConfig: Record<string, string> | null = null;

export async function getConfig(): Promise<Record<string, string>> {
  if (cachedConfig) return cachedConfig;
  cachedConfig = await (window as any).appConfig.get();
  return cachedConfig;
}

export async function getApiUrl(): Promise<string> {
  const cfg = await getConfig();
  return cfg.API_URL || 'http://localhost:8000';
}

// anywhere in app code
import { getApiUrl } from '@lib/config';
const res = await fetch(`${await getApiUrl()}/api/patients/`);

```

Important: Do **not** prefix these with `NEXT_PUBLIC_`. They are not env vars from Next.js's perspective — they're runtime values fetched over IPC, which is the whole point.

Task 8.4 — In-App Settings Screen

```

// Settings page
await window.appConfig.set({ API_URL: 'http://10.0.0.5:8000' });
// Then prompt: "Restart now to apply changes?"

```

Task 8.5 — Build the Frontend

- **Option A (recommended): Next.js static export** (output: 'export'), served by Electron via `file:///`. No Node server runs on the client; the licensing gate lives in the Electron main process instead of Next middleware.
- **Option B: Next.js standalone**, started as a child process from Electron. Heavier, gives SSR/server actions on the client — usually overkill here.

```
// next.config.js
module.exports = {
  output: 'export',
  images: { unoptimized: true },
};

npm run build      # produces out/
```

Task 8.6 — Package With electron-builder

```
npm install --save-dev electron-builder dotenv

{
  "main": "electron/main.js",
  "build": {
    "appId": "com.yourcompany.appclient",
    "productName": "App Client",
    "files": ["electron/**/*", "out/**/*"],
    "extraResources": [{ "from": "default.env", "to": "default.env" }],
    "win": { "target": "nsis", "icon": "build/icon.ico" },
    "nsis": {
      "oneClick": true,
      "perMachine": false,
      "allowToChangeInstallationDirectory": false
    }
  }
}

npx electron-builder --win
```

Produces `dist/AppClient-Setup-1.0.0.exe` — one-click install, no admin required (`perMachine: false` installs to `%LOCALAPPDATA%`).

Task 8.7 — Test the Packaged App

```
✓ App installs without admin prompt
✓ App starts and reaches the backend at the IP in config.env
✓ Editing config.env + restarting picks up the new API_URL
✓ Logs land in %APPDATA%\AppClient\logs\
✓ License-invalid screen shows the right message for each failure reason
✓ Retry button re-checks and opens the main window once the license is valid again
```

STOP

```
✓ Electron app packaged as one-click installer
```

- ✓ Runtime config.env editable post-build (file edit or in-app Settings)
 - ✓ NEXT_PUBLIC_* is NOT used for the API URL – values are read via IPC at runtime
 - ✓ App reaches the real compiled backend on the LAN
 - ✓ license-invalid.html exists and renders correctly, with a working retry path
-

Phase 9 — Frontend Licensing Gate in Electron

The Electron main process calls the backend's `/api/license/status/` endpoint **before** opening the main window (already wired in Phase 8's `startApp()`). If the license is invalid, it opens the `license-invalid.html` window with the reason and a working retry button instead of the app.

The renderer doesn't need to know about licensing at all — if the main window opened, the license was valid. Backend API calls will also return 401/403 if the license is revoked mid-session, which the renderer handles as a generic auth error and shows a "session expired" dialog.

Task 9.1 — Full-Stack Lifecycle Test

- ✓ valid license → main window opens, app runs normally
- ✓ missing license file → license-invalid window with reason="missing"
- ✓ expired license → license-invalid window with reason="expired"
- ✓ license for wrong hwid → license-invalid window with reason="hwid_mismatch"
- ✓ tampered license file → license-invalid window with reason="tampered"
- ✓ backend unreachable → license-invalid window with reason="unreachable"
- ✓ (online mode) revoked → blocked after grace period, frontend reflects it
- ✓ license revoked mid-session → next API call returns 403, renderer shows session-expired dialog
- ✓ fixing the license server-side, then clicking Retry → app opens without a restart

Task 9.2 — Multi-Client Test

Install the Electron app on **3 different client PCs** on the same LAN, all pointing at the same backend:

- ✓ All three can log in and use the app concurrently
 - ✓ If `max_clients` is set in the license and exceeded, the 4th client gets a clear "client limit reached" message
-

STOP

- ✓ Electron client reflects license status correctly for every backend state
 - ✓ Multiple client PCs work concurrently against the same backend
-

Phase 10 — Two Installers

Server installer (`app-server-setup.exe`)

Bundles:

- Compiled backend binary (`backend.exe` / `backend.bin`)
- `default.env` template
- `public_key.pem`
- `scripts/get_machine_id.py`, `scripts/backup.sh` or `.bat`
- NSSM service registration (Windows) or `systemd` unit (Linux)
- Firewall rule for the chosen `PORT`

On first run, the installer:

1. Asks for install directory (default `C:\AppServer\` or `/opt/app/app-server`).
2. Runs `get_machine_id.py` automatically and displays the HWID with a "Copy" button, so the operator can send it to you immediately (closes the loop from Task 4.3b without extra manual steps).
3. Asks the operator to drop `license.lic` into `config\` (or pastes it during install, if you already have it back from step 2 for a repeat install).
4. Registers and starts the service.
5. Verifies `/api/license/status/` returns `valid: true`.

Client installer (`app-client-setup.exe`)

Produced by `electron-builder` in Phase 8. Bundles:

- The Electron app + Next.js static export.
- `default.env` template (copied to `%APPDATA%\AppClient\config.env` on first launch).

On first run, the app:

1. Asks the user for the **server IP** (or auto-discovers via mDNS, optional).
2. Writes `API_URL=http://<server-ip>:<port>` to `config.env`.
3. Restarts and connects.

One-click from the user's perspective: `install` → enter server IP → `use`.

Distribution

- Server installer: delivered once to IT (or installed by you on-site).
- Client installer: distributed via shared network drive, USB stick, or a download link on the

internal network. Each workstation runs it independently.

STOP

✓ Server installer: one machine, registers service, license validates, HWID hand-off works
✓ Client installer: any number of machines, each with its own config.env
✓ Zero-touch from server side for additional client installs

Phase 11 — Security Hardening (Audit Pass)

Most of these were already implemented earlier ("do this now, not at the end" — Phase 1). This phase is a **verification pass**, not new construction — go back through and confirm nothing regressed.

Backend (server)

- `DEBUG=False` — confirm in the live `.env`, not just the template.
- Secure, unique `SECRET_KEY` per deployment — never the same key across two clients.
- Firewall restricted to the app's `PORT` (Task 2.6) — no other ports exposed.
- `data/db.sqlite3` permissions restricted to the service account.
- HTTPS via the Nginx reverse proxy from Phase 2 Task 2.3, if the LAN spans untrusted segments.
- Confirm `config/.env` and `config/license.lic` are still `chmod 600`, owned by the service account — **re-check after every license re-issue, backup restore, or fresh install**, since those operations often recreate the file with default permissions.
- Online mode: phone-home calls must be HTTPS, and the license-server endpoint should be rate-limited against HWID-guessing.
- CORS: `CORS_ALLOWED_ORIGINS` locked to the exact Electron origin, never `"*"`.

Frontend (client PCs)

- `contextIsolation: true, nodeIntegration: false` — confirm on **both** the main window and the license-invalid window (v4 only had it on the main window).
- `webSecurity: true` (default — do not disable).
- Set a `Content-Security-Policy` in the renderer's HTML to restrict fetch origins to the configured `API_URL`.
- Don't log `config.env` contents to disk.
- If using `electron-updater`: verify update signatures (`publisherName` in `electron-builder` config).

- `config.env` in `%APPDATA%` is readable by that user only by default — don't weaken it.
-

Milestone Order

1. **Phase 1–2** — backend runs correctly, uncompiled, reachable from another machine, CORS/ALLOWED_HOSTS driven by `.env`.
2. **Phase 3** — logging wired up, so every later phase has somewhere to write failures.
3. **Phase 4** — backend licensing enforced and fully tested, still uncompiled, with the HWID hand-off process defined.
4. **Phase 5** — compile the backend with Nuitka, re-test licensing against the compiled binary.
5. **Phase 6** — wrap the *compiled* binary in a systemd/NSSM service.
6. **Phase 7** — backups wired around the now-live, serviced deployment. **Server track ends here.**
7. **Phase 8** — Electron frontend built, packaged, runtime `config.env` working, license-invalid screen and retry path functional.
8. **Phase 9** — frontend license gate, multi-client test.
9. **Phase 10** — two separate installers produced and tested end-to-end.
10. **Phase 11** — hardening audit on both sides.

Each phase's STOP checkpoint should pass before starting the next one. Backend and frontend remain completely independent artifacts with their own installers, their own config files, and their own update cycles — the frontend's config is editable post-build via `config.env` in `userData` + IPC, never via `NEXT_PUBLIC_*` env vars.

Future Feature — License Renewal Endpoint

This applies to **any** project built on this template, not just one product line — anywhere you've shipped this Django + Electron + JWT licensing scaffold, this feature drops in the same way.

The problem

Right now, renewing an expired or changed license means: generate a new `license.lic` → send it to the customer/IT → someone manually overwrites `config/license.lic` on the server → restart the service. That's slow, error-prone, and doesn't scale past a handful of deployments.

Two ways to solve it

A. Push model — an authenticated renewal endpoint

Add `POST /api/license/renew/` that accepts a new signed token and applies it live, without

a manual file swap or service restart.

Design constraints:

- This endpoint must be **exempt from the license-required middleware** — a genuinely expired customer still needs to be able to renew.
- It must **not** rely on normal user auth (which may itself be gated by the expired license). Use a separate static admin token stored in `.env` (`LICENSE_ADMIN_TOKEN`), or a dedicated Django superuser check.
- It must validate the incoming token exactly like startup does (signature, HWID) before writing anything.
- It must write atomically (temp file + `os.replace`) so a crash mid-write never leaves `license.lic` corrupted.
- It should log the renewal to `license.log`.

```
# views.py
import os, tempfile, logging
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAdminUser
from rest_framework.response import Response

from .license_check import validate_license_token, current_hwid, get_public_key,
validate_license

logger = logging.getLogger("license")

@api_view(["POST"])
@permission_classes([IsAdminUser])
def renew_license(request):
    new_token = request.data.get("license_token", "").strip()
    if not new_token:
        return Response({"valid": False, "reason": "missing_token"}, status=400)

    public_key = get_public_key(CONFIG_DIR)
    try:
        payload = validate_license_token(new_token, public_key)
    except Exception:
        return Response({"valid": False, "reason": "tampered"}, status=400)

    if payload["hwid"] != current_hwid():
        return Response({"valid": False, "reason": "hwid_mismatch"}, status=400)

    # Guard against accidentally installing an unrelated deployment's license
    try:
        current = validate_license(CONFIG_DIR)
        if current["license_id"] != payload["license_id"] and not
request.data.get("force"):
        return Response({"valid": False, "reason": "license_id_mismatch"},
status=400)
    except Exception:
        pass # no valid current license – first-time install via this endpoint
is fine

    license_path = os.path.join(CONFIG_DIR, "license.lic")
    fd, tmp_path = tempfile.mkstemp(dir=CONFIG_DIR)
    with os.fdopen(fd, "w") as f:
        f.write(new_token)
```

```

os.replace(tmp_path, license_path)
os.chmod(license_path, 0o600)

logger.info(f"License renewed via API: {payload['license_id']}
exp={payload['exp']}")
return Response({
    "valid": True,
    "client": payload["client"],
    "exp": payload["exp"],
    "features": payload["features"],
})

# urls.py – exempt from the license-required middleware alongside
/api/license/status/
urlpatterns = [
    path("api/license/status/", license_status_view),
    path("api/license/renew/", renew_license),
]

```

B. Pull model — automatic renewal for online-mode deployments

For deployments running in "mode": "online" (Task 4.6), instead of waiting for someone to push a new token, have the backend check for one itself:

- A scheduled job (systemd timer, Windows Task Scheduler, or an APScheduler job inside Django) calls `payload["server_url"]` with `license_id` + `hwid` on a daily interval.
- If the license server returns a newer signed token, run it through the **same validation and atomic-write path** as the push endpoint above — factor that into a shared `apply_new_license(token)` function so both code paths stay in sync.
- If the license server is unreachable, fall back to the existing `grace_days` handling — no change needed there.

```

# license_renewal.py
def apply_new_license(token, config_dir=CONFIG_DIR):
    """Shared by both the push endpoint and the pull/cron job."""
    public_key = get_public_key(config_dir)
    payload = validate_license_token(token, public_key) # raises on bad
signature/expiry
    if payload["hwid"] != current_hwid():
        raise ValueError("hwid_mismatch")

    license_path = os.path.join(config_dir, "license.lic")
    fd, tmp_path = tempfile.mkstemp(dir=config_dir)
    with os.fdopen(fd, "w") as f:
        f.write(token)
    os.replace(tmp_path, license_path)
    os.chmod(license_path, 0o600)
    logger.info(f"License renewed: {payload['license_id']}
exp={payload['exp']}")
    return payload

```

Frontend hook-up

Add a "Renew License" action to the Electron Settings screen (Task 8.4). It posts the new token to `/api/license/renew/`, then calls the existing `retryLicenseCheck()` IPC handler from

Phase 8 — the app reflects the renewed state immediately, no restart needed:

```
async function renewLicense(token: string) {
  const apiUrl = await getApiUrl();
  const res = await fetch(`${apiUrl}/api/license/renew/`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${adminToken}` },
    body: JSON.stringify({ license_token: token }),
  });
  const result = await res.json();
  if (result.valid) {
    await window.appConfig.retryLicenseCheck();
  }
  return result;
}
```

Test checklist

```
✓ /api/license/renew/ works even while the currently-installed license is expired
✓ endpoint rejects a token signed with the wrong key
✓ endpoint rejects a token whose hwid doesn't match this machine
✓ endpoint rejects a token for a different license_id unless force=true is passed
✓ a crash mid-write never leaves license.lic empty or corrupted (atomic replace holds)
✓ renewal is logged to license.log with the new license_id and exp
✓ (pull model) scheduled job picks up a newer token automatically within one interval
✓ (pull model) license server unreachable → falls back to grace_days, no crash
✓ Electron "Renew License" action updates the app state without a restart
```